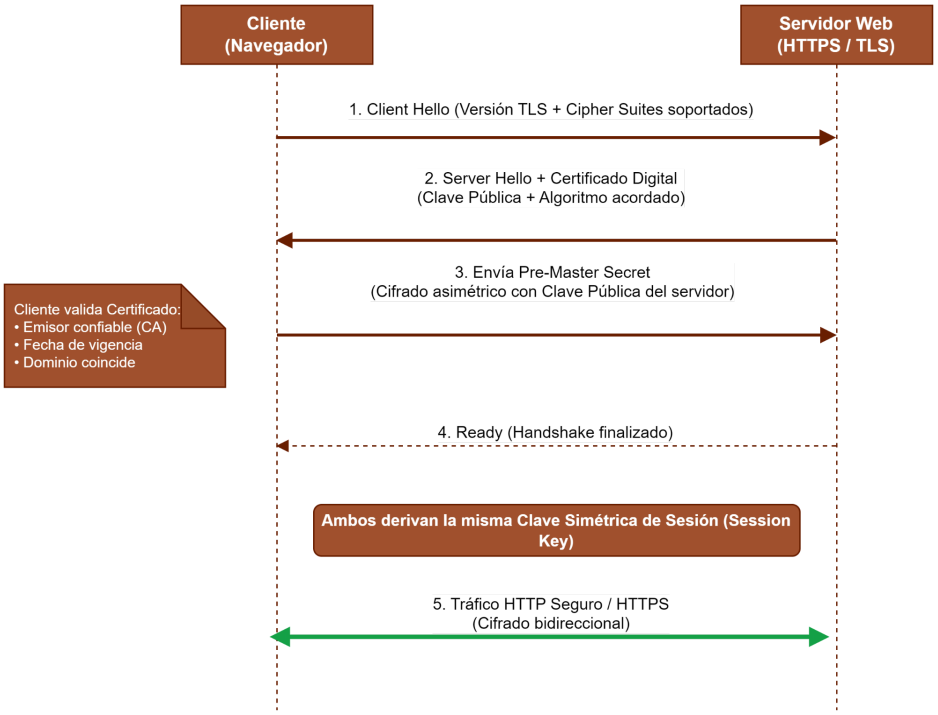
b) Investigación:

- Certificados de Seguridad SSL (Secure Sockets Layer):
    - ¿Qué es SSL, para que sirve y cómo funciona? ¿Cómo se identifica visualmente un navegador que se está utilizando SSL? Realizar una gráfica del funcionamiento. ¿Cuál es la diferencia entre HTTP y HTTPS? ¿Cómo puede obtener un certificado SSL gratuito?
- Json Web Token (JWT):
    - ¿Qué es JWT (Json Web Token) y cómo funciona? Realizar una gráfica del funcionamiento.
- AUTH0:
    - ¿Qué es AUTH0 y cómo funciona? Realizar una gráfica del funcionamiento.
- API´s Seguras:
    - ¿Cuál es la importancia de que nuestras API's sean seguras? Detallar las buenas prácticas de programación, respecto de seguridad, que se deben tener en cuenta al momento de desarrollar una API.

- **Certificados de Seguridad SSL (Secure Sockets Layer):**
- SSL es un protocolo criptográfico diseñado para autenticar la identidad de un servidor y cifrar los canales de comunicación en redes como Internet. Su evolución moderna es TLS (Transport Layer Security), aunque coloquialmente se sigue llamando SSL. Sirve para asegurar que la información sensible viaje de extremo a extremo sin ser interceptada ni alterada por terceros.
    - Funcionamiento básico (SSL/TLS Handshake):
        - Negociación inicial (Client Hello / Server Hello): El navegador y el servidor acuerdan la versión del protocolo y el conjunto de algoritmos de cifrado (cipher suites).
        - Validación del certificado y clave pública: El servidor envía su certificado digital emitido por una Autoridad Certificadora (CA). El cliente verifica su validez y extrae la clave pública.
        - Criptografía asimétrica para intercambio: El cliente genera una clave de sesión secreta precursora (pre-master secret), la cifra con la clave pública del servidor y se la envía. El servidor la descifra con su clave privada correspondiente.
        - Cifrado simétrico de datos: Ambos derivan una clave de sesión compartida y el resto del tráfico se transfiere mediante cifrado simétrico, el cual es mucho más rápido y consume menos cómputo.

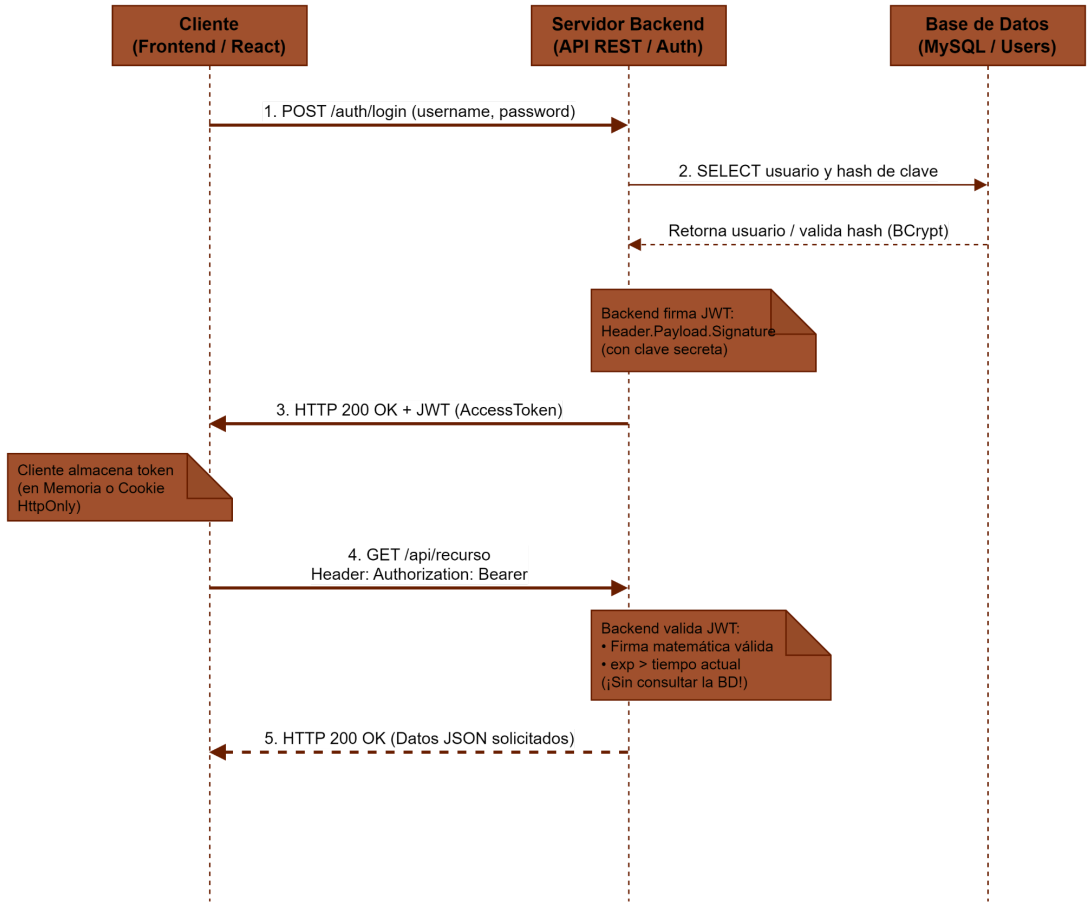


-

- Identificación visual en el navegador:
  - En la barra de direcciones se muestra el prefijo https://.
  - Se visualiza el ícono de un candado cerrado (en navegadores modernos suele integrarse en un menú de ajustes de sitio que confirma "La conexión es segura").
  - Al hacer clic sobre el ícono, se despliegan los detalles del certificado: dominio verificado, emisor (CA) y fecha de expiración.
  - Si el certificado falla o está ausente, el navegador muestra advertencias a pantalla completa como "La conexión no es privada" (NET::ERR_CERT_COMMON_NAME_INVALID).
- Diferencia entre HTTP y HTTPS:
  - HTTP (HyperText Transfer Protocol): Opera por defecto en el puerto 80. Transmite los datos en texto plano. Cualquier atacante en la red puede leer o manipular las solicitudes mediante ataques de tipo Man-in-the-Middle (MitM).
  - HTTPS (HTTP Secure): Opera por defecto en el puerto 443. Es el protocolo HTTP encapsulado sobre una capa SSL/TLS. Garantiza confidencialidad (los datos van cifrados), integridad (no se modifican en tránsito) y autenticación (se valida la identidad del servidor).
- Cómo obtener un certificado SSL gratuito:
  - Let's Encrypt: Es una Autoridad Certificadora abierta y gratuita promovida por la Internet Security Research Group (ISRG). Permite obtener certificados SSL/TLS automáticos utilizando clientes como Certbot vía protocolo ACME.

    - Proveedores de nube / CDN: Plataformas como Cloudflare, AWS (vía AWS Certificate Manager) o servicios de hosting como Vercel y Firebase Hosting emiten y renuevan certificados SSL gratuitos de forma automatizada al vincular un dominio personalizado.

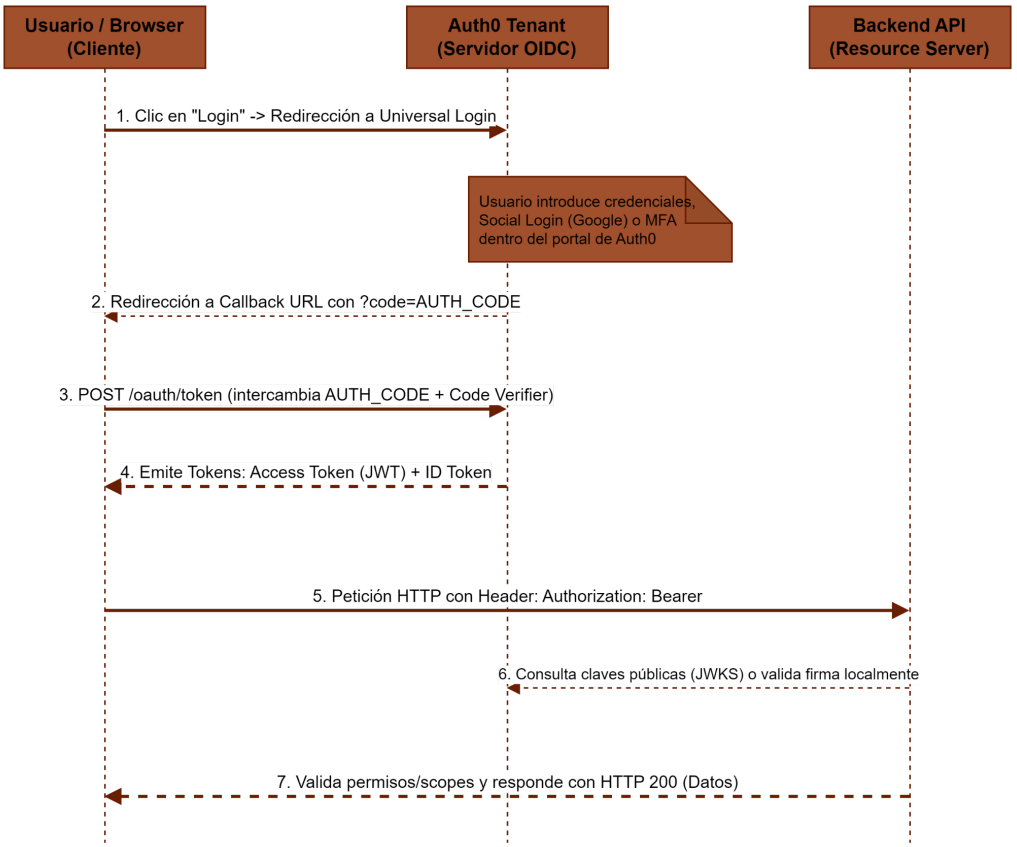
- **Json Web Token (JWT):**
    - Es un estándar abierto compacto y autónomo para transmitir información de forma segura entre partes como un objeto JSON. Al estar firmado digitalmente, su contenido puede ser verificado y confiado.
- Estructura del token: Se compone de tres partes codificadas en Base64Url separadas por puntos (Header.Payload.Signature):
    - Header: Define el tipo de token (JWT) y el algoritmo de firma usado (HS256, RS256).
    - Payload: Contiene los claims (declaraciones sobre la entidad, como sub, name, roles, exp). No debe contener datos confidenciales sin cifrar, ya que solo está codificado en Base64.
    - Signature: Se calcula tomando el Header en Base64, el Payload en Base64, un secreto del servidor y pasándolos por el algoritmo de firma. Previene la alteración de los datos.
- Funcionamiento:
    - El usuario envía sus credenciales al endpoint de autenticación (/auth/login).
    - El servidor valida las credenciales en su base de datos. Si son válidas, genera el JWT firmado con una clave secreta y lo devuelve al cliente.
    - El cliente almacena el token (usualmente en memoria o en una cookie HttpOnly) y lo adjunta en las siguientes peticiones HTTP dentro de la cabecera Authorization: Bearer <token>.
- El servidor valida la firma del token con su secreto/clave pública sin necesidad de consultar la base de datos de sesiones (autenticación stateless). Si la firma y la fecha de expiración son válidas, autoriza la petición.



-

- **Auth0:**
- es una plataforma de Identidad como Servicio (IDaaS - Identity-as-a-Service) que externaliza la autenticación y autorización de sistemas. Implementa estándares universales de la industria como OAuth 2.0, OpenID Connect (OIDC) y SAML, permitiendo integrar inicio de sesión con redes sociales (Google, GitHub), autenticación multifactor (MFA), Single Sign-On (SSO) y control de acceso basado en roles (RBAC) sin tener que programar la infraestructura de usuarios desde cero.
- Funcionamiento (Flujo de Código de Autorización con PKCE):
    - El usuario hace clic en "Iniciar Sesión" en la aplicación.
    - La app redirige al usuario hacia el servidor de autenticación alojado por Auth0 (Universal Login).
    - El usuario se autentica en Auth0 (usuario/contraseña, Google, MFA, etc.).
    - Auth0 valida la identidad y redirige de vuelta al navegador con un código de autorización de un solo uso.
    - El cliente/servidor intercambia ese código de autorización con Auth0 a cambio de tokens: un ID Token (datos del usuario para el frontend) y un Access Token (JWT para consumir APIs backend).
    - La aplicación realiza solicitudes al backend enviando el Access Token en el encabezado Authorization: Bearer.

- El backend valida la firma criptográfica del token consultando periódicamente la clave pública de Auth0 (vía JWKS: JSON Web Key Set).



-

- **Api's seguras:**
- Las APIs conectan directamente las interfaces de usuario (web, móviles, servicios externos) con la lógica de negocio y las bases de datos de una organización. Una falla de seguridad en este nivel deja expuesta toda la persistencia: robo de credenciales, manipulación masiva de registros, filtración de datos de clientes, denegación de servicios o compromisos legales severos de privacidad de datos.
- Buenas prácticas de programación y diseño seguro (Top OWASP API Security)
    - Autenticación y Autorización robustas:
        - Usar tokens de corta duración (Access Token JWT firmado) combinados con Refresh Tokens rotativos.
        - Implementar control de acceso a nivel de objeto (BOLA / IDOR mitigation): verificar siempre que el usuario autenticado sea el dueño legítimo del recurso que intenta modificar (/pedidos/{id}).
        - Implementar control de acceso basado en roles (RBAC) o atributos (ABAC).
    - Cifrado en tránsito y en reposo:
        - Forzar HTTPS (TLS 1.3) en todos los endpoints sin excepción, deshabilitando conexiones HTTP no seguras.

    - Cifrar campos sensibles en la base de datos (contraseñas con algoritmos lentos como BCrypt, Argon2 o PBKDF2).
- Validación, saneamiento y tipado de entradas:
    - Validar estrictamente esquemas de entrada (longitud, tipos de datos, patrones regex, campos requeridos) mediante DTOs para evitar Mass Assignment y sobreescrituras indebidas.
    - Usar consultas parametrizadas (ORM con JPA/Hibernate o PreparedStatements) para anular inyecciones SQL.
- Limitación de tasa (Rate Limiting) y Cuotas:
    - Proteger endpoints sensibles (login, registro, recuperación de contraseña) contra ataques de fuerza bruta y DoS limitando la cantidad de peticiones permitidas por IP o usuario por minuto.
- Manejo seguro de errores y no exposición de trazas:
    - No enviar al cliente trazas de excepción del servidor (stack traces de Java/Spring), versiones del motor de base de datos o consultas internas.
    - Usar controladores globales de excepciones para devolver respuestas estandarizadas y genéricas con códigos HTTP precisos (400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found).
- Configuración restrictiva de CORS (Cross-Origin Resource Sharing):
    - Limitar explícitamente los orígenes permitidos en @CrossOrigin al dominio del frontend autorizado, evitando el uso de comodines permisivos ("*") en producción cuando se manejan credenciales o cookies.
- Paginación y control de recursos:
    - Limitar el número máximo de registros devueltos por consulta (por ejemplo, exigir limit=50&page=1) para prevenir el agotamiento de memoria del servidor por peticiones masivas.